Plano Mestre: Sistema de Depósito de Materiais de Construção (Python & Tkinter)

Prazo: 13/10 a 29/10 (16 dias)
Persistência: Arquivos (JSON)
Interface Gráfica: Tkinter / TTK

1. Mapeamento dos Requisitos Obrigatórios da Avaliação (Garantia de Nota 10,0)

Cada requisito de código solicitado pelo professor será atendido de forma explícita e documentada:

Estrutura Obrigatória	Pontuação	Onde e Como Será Aplicada no Código
`if/else` e/ou `elif`	1,0	Validação de formulários (campos vazios, preço negativo), checagem de estoque mínimo, verificação de existência do produto.
`while` e/ou `for`	1,0	Varredura de listas de produtos para montagem de relatórios, busca textual por filtros, iteração sobre itens da tabela.
Subprogramas (Funções / Métodos)	1,0	Modularização em funções com responsabilidade única: `carregar_estoque()`, `salvar_estoque()`, `calcular_total_patrimonio()`, `validar_dados()`.
Listas / Matrizes	1,0	Lista de tuplas para inserção direta no `Treeview` (matriz de linhas e colunas) e histórico de movimentações (entradas/saídas).
Strings / Manipulação de Texto	1,0	Tratamento de texto com `.strip()`, `.upper()` / `.title()`, formatação de moeda `f"R$ {valor:.2f}"`, busca insensível a maiúsculas (`in`).
Registros / Dicionários / Tuplas	1,0	**Dicionários**: Cada material é um dicionário (`{"id": ..., "nome": ..., "qtd": ..., "preco": ...}`). **Tuplas**: Configurações imutáveis de categorias e colunas da tabela.
Arquivos (Persistência)	1,0	Uso de `json.dump()` e `json.load()` com `open('dados_materiais.json', 'r'/'w', encoding='utf-8')`.
Engenharia de Software & Apresentação	3,0	Requisitos Funcionais/Não-Funcionais, Histórias de Usuário, Casos de Uso, Diagrama e Roteiro da Apresentação.

2. Engenharia de Software: Requisitos, Histórias e Casos de Uso

2.1. Demandas Encontradas no Estabelecimento

- Dificuldade de controlar o estoque físico de materiais volumosos e fracionados (cimento, areia, tubos, tijolos).
- Falta de controle sobre quais produtos atingiram o estoque crítico (risco de falta de mercadoria para venda).
- Falta de um cálculo rápido do valor financeiro imobilizado no pátio da loja.
- Perda de tempo ao buscar preços e disponibilidade no balcão de atendimento.

2.2. Requisitos Funcionais (RF)

- **RF01:** Cadastrar novo material com código identificador, nome, categoria, quantidade, quantidade mínima de segurança e preço unitário.
- **RF02:** Listar todos os materiais cadastrados em uma tabela organizada por colunas.
- **RF03:** Registrar entrada e saída (venda/baixa) de materiais, atualizando a quantidade em estoque.
- **RF04:** Impedir saídas que resultem em estoque negativo.
- **RF05:** Filtrar/pesquisar materiais por nome ou categoria em tempo real.
- **RF06:** Emitir alerta visual para produtos com estoque igual ou inferior à quantidade mínima.
- **RF07:** Exibir resumo financeiro (valor total do patrimônio em estoque e total de itens).
- **RF08:** Salvar e carregar automaticamente todos os dados em arquivo local (`JSON`).

2.3. Requisitos Não-Funcionais (RNF)

- **RNF01 (Usabilidade):** Interface gráfica intuitiva com janela de fácil operação para balconistas e estoquistas via Tkinter.
- **RNF02 (Portabilidade):** O sistema deve rodar em qualquer computador com Python 3 instalado, sem necessidade de instalar servidores de banco de dados pesados.
- **RNF03 (Confiabilidade):** Persistência segura em arquivo com codificação UTF-8 e tratamento de falhas caso o arquivo não exista ou esteja corrompido.

2.4. Histórias de Usuário (User Stories)

1. **Como balconista**, quero buscar um material pelo nome para informar rapidamente ao cliente se há pronta-entrega e o valor unitário.
2. **Como gerente do depósito**, quero registrar a chegada de caminhões com mercadorias para atualizar a quantidade disponível no sistema.
3. **Como estoquista**, quero ser avisado quando um material estiver com poucas unidades para solicitar compra ao fornecedor antes que acabe.
4. **Como proprietário**, quero ver o valor total em reais que tenho em estoque para acompanhar meu capital de giro.

2.5. Diagrama de Casos de Uso (Mermaid)

```
```

3. Cronograma Passo a Passo (13/10 a 29/10)

```
[13-16/10: Fundação Lógica] ➔ [17-20/10: Interface Tkinter] ➔ [21-24/10: Integração e Regras] ➔ [25-28/10: Documentos e Apresentação] ➔ [29/10: Entrega]
```

🗓️ Semana 1: Estrutura de Dados e Lógica Pura (13/10 a 18/10)

- **Dia 1 (13/10 - Hoje):** Alinhamento dos requisitos, definição da estrutura de dados dos materiais e criação do plano.
- **Dia 2 (14/10):** Criar o módulo de lógica pura (`gerenciador_estoque.py`):
  - Dicionários para produtos e Tuplas para categorias imutáveis.
  - Funções com `if/elif/else` para validação de dados.
- **Dia 3 (15/10):** Manipulação de arquivos (`JSON`):
  - Funções de carregar e salvar com tratamento de erros (`try/except`).
- **Dia 4 (16/10):** Estruturas de repetição (`for` e `while`):
  - Função de busca/filtro por substrings (`.lower()`).
  - Cálculo de totais acumulados com `for` .
- **Dias 5 e 6 (17 e 18/10):** Introdução e montagem da janela visual (`interface.py`):
  - Janela principal, formulários de entrada (`Entry`), botões (`Button`) e painel de avisos.

🗓️ Semana 2: Interface Avançada e Integração (19/10 a 24/10)

- **Dia 7 (19/10):** Tabela de dados (`ttk.Treeview`):
  - População da tabela usando matrizes/listas de linhas.
- **Dia 8 (20/10):** Conectar botões da interface com as funções do arquivo de lógica.
- **Dia 9 (21/10):** Sistema de alerta de estoque crítico (destaque em cores ou caixas de mensagem `messagebox`).
- **Dia 10 (22/10):** Modal/janela de movimentação rápida (Entrada de Carga e Saída/Venda).
- **Dias 11 e 12 (23 e 24/10):** Barra de pesquisa em tempo real e limpeza/organização do código com comentários didáticos.

🗓️ Semana 3: Testes, Relatório e Preparação para Defesa (25/10 a 28/10)

- **Dia 13 (25/10):** Testes rigorosos de usuário (tentar quebrar o sistema com entradas inválidas e corrigir).

- **Dia 14 (26/10):** Redação do documento final de entrega (Requisitos, Casos de Uso e prints das telas).
- **Dia 15 (27/10):** Mapeamento linha a linha de onde cada estrutura solicitada pelo professor foi aplicada.
- **Dia 16 (28/10):** Ensaio das respostas para a apresentação.
- **29/10 (Dia D):** Envio do projeto e apresentação com tranquilidade!

4. Guia de Respostas para a Apresentação Oral

Quando o professor fizer as perguntas da avaliação, você responderá com base neste roteiro:

1. **Quais foram as demandas encontradas no estabelecimento para o projeto?**

   > *"Identificamos descontrole no estoque físico de materiais de construção, risco de ruptura em itens essenciais (como cimento e argamassa), lentidão no atendimento ao cliente para checar preços e ausência de relatórios sobre o valor financeiro total imobilizado no depósito."*

2. **Quais foram os objetivos traçados para o seu projeto?**

   > *"Criar uma aplicação desktop rápida, leve e visual que centralize o cadastro de materiais, registre entradas e saídas em tempo real, forneça alertas visuais de reposição e mantenha todos os dados salvos com persistência em arquivos."*

3. **Quais os requisitos desenvolvidos para o seu projeto?**

   > *"Desenvolvemos requisitos de cadastro completo, busca inteligente por categoria e nome, controle de estoque com impedimento de saldo negativo, alerta de estoque mínimo e painel com patrimônio financeiro total acumulado."*

4. **O que foi atendido pelo seu projeto?**

   > *"O MVP atendeu 100% dos requisitos funcionais e não-funcionais planejados, além de cumprir integralmente todas as estruturas de programação exigidas no edital de avaliação."*

5. **Quais e onde foram utilizadas as estruturas no código?**

- **`if/elif/else`:** Na validação de preços/quantidades e na verificação de estoque mínimo.
- **`for` e `while`:** Na varredura dos dados para preenchimento da tabela gráfica e no cálculo do patrimônio total.
- **Subprogramas:** Em funções modulares de persistência, cálculo e manipulação de estado.
- **Matrizes e Listas:** Na conversão dos registros para linhas/colunas do componente `Treeview`.
- **Strings:** No tratamento de entradas (`.strip()`, `.upper()`) e formatação financeira (`f"R$ {val:.2f}"`).
- **Dicionários e Tuplas:** Dicionários para modelar as propriedades do material e tuplas para definir colunas e categorias imutáveis.
- **Arquivos:** No carregamento e gravação contínua no arquivo `dados_materiais.json`.